

NAGALAND UNIVERSITY

(A Central University Established by an Act of Parliament 1989)

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Kohima Campus, Nagaland – 797004



Assignment 4

ITB301: Object Oriented Programming

Submitted By:

Soushnabh Bagchi

Roll / ID: 2505264

Semester: 3rd Sem

Submitted To:

Er Sudipto Patowary

Assistant Professor

Dept. of IT

Date: September 30, 2026

1. Create a class Array containing an integer pointer data and an integer size. Allocate memory dynamically in the constructor and implement constructor methods.

Code:

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Array {
6  public:
7      int *data;
8      int size;
9      // Parameterized Constructor
10     Array(int size) {
11         this->size = size;
12         data = new int[size];
13     }
14
15     // Copy Constructor
16     Array(const Array &a) {
17         size = a.size;
18         data = new int[size];
19         for (int i = 0; i < size; i++) {
20             data[i] = a.data[i];
21         }
22     }
23
24     // Destructor
25     ~Array() { delete[] data; }
26
27     // Display
28     void display() {
29         cout << '[';
30         for (int i = 0; i < size; i++) {
31             if (i > 0) {
32                 cout << ", ";
33             }
34             cout << data[i];
35         }
36         cout << ']' << endl;
37     }
```

C++

```

38 };
39
40 int main() {
41     int n;
42     cout << "Enter the array length: ";
43     cin >> n;
44     Array a(n);
45     Array b = a;
46     a.display();
47     b.display();
48     return 0;
49 }

```

Output:

```

1 Enter the array length: 5
2 [0, 0, 0, 0, 0]
3 [0, 0, 0, 0, 0]

```

2. Create a class Buffer containing a dynamically allocated array and implement constructor methods.**Code:**

```

1  #include <iostream>
2  #include <utility>
3
4  using namespace std;
5
6  class Buffer {
7  public:
8      int *data;
9      int size;
10     // Parameterized Constructor
11     Buffer(int size) {
12         this->size = size;
13         data = new int[size];
14     }
15
16     // Copy Constructor
17     Buffer(const Buffer &a) {
18         size = a.size;
19         data = new int[size];

```

C++

```

20     for (int i = 0; i < size; i++) {
21         data[i] = a.data[i];
22     }
23 }
24
25 // Move Constructor
26 Buffer(Buffer &&a) {
27     size = a.size;
28     a.size = 0;
29     data = a.data;
30     a.data = nullptr;
31 }
32
33 // Destructor
34 ~Buffer() { delete[] data; }
35 };
36
37 int main() {
38     int n;
39     cout << "Enter the Buffer length: ";
40     cin >> n;
41     Buffer a(n);
42     Buffer b = a;
43     Buffer c = std::move(b);
44     return 0;
45 }

```

Output:

```

1 Enter the Buffer length: 10
2
3 ▶ Task `Compile & Run C++` finished successfully

```

3. Create a class Person and implement constructor methods.**Code:**

```

1 #include <iostream>
2 #include <utility>
3
4 using namespace std;
5
6 class Person {

```

C++

```
7  public:
8      string name;
9      int age;
10     // Parameterized Constructor
11     Person(string name, int age) {
12         this->name = name;
13         this->age = age;
14     }
15
16     // Copy Constructor
17     Person(const Person &a) {
18         name = a.name;
19         age = a.age;
20     }
21
22     // Move Constructor
23     Person(Person &&a) {
24         age = a.age;
25         a.age = 0;
26         name = std::move(a.name);
27     }
28 };
29
30 int main() {
31     string firstname, lastname;
32     int age;
33     cout << "Enter the name: ";
34     cin >> firstname >> lastname;
35     string name = firstname + ' ' + lastname;
36     cout << "Enter the age: ";
37     cin >> age;
38     Person a(name, age);
39     Person b = a;
40     Person c = std::move(b);
41     return 0;
42 }
```

Output:

```
1 Enter the name: Soushnabh Bagchi
2 Enter the age: 18
3
```

4 ▶ Task `Compile & Run C++` finished successfully

4. Write a C++ program to create a dynamic integer array using a constructor.

Code:

```
1  #include <iostream>
2
3  using namespace std;
4
5  class DynamicArray {
6  public:
7      int *data;
8      int size;
9      DynamicArray(int size) {
10         this->size = size;
11         data = new int[size];
12     }
13
14     ~DynamicArray() { delete[] data; }
15
16     void input() {
17         cout << "Enter the elements: ";
18         for (int i = 0; i < size; i++) {
19             cin >> data[i];
20         }
21     }
22
23     void display() {
24         cout << '[';
25         for (int i = 0; i < size; i++) {
26             if (i > 0) {
27                 cout << ", ";
28             }
29             cout << data[i];
30         }
31         cout << ']' << endl;
32     }
33 };
34
35 int main() {
```

C++

```

36  int n;
37  cout << "Enter the DynamicArray length: ";
38  cin >> n;
39  DynamicArray a(n);
40  a.input();
41  a.display();
42  return 0;
43 }

```

Output:

```

1 Enter the DynamicArray length: 4
2 Enter the elements: 2 4 6 8
3 [2, 4, 6, 8]

```

5. Write a C++ program to multiply two polynomials using dynamic arrays and a constructor.**Code:**

```

1  #include <iostream>
2
3  using namespace std;
4
5  struct monomial {
6      int coeff;
7      int power;
8  };
9
10 class DynamicArray {
11 public:
12     monomial *data;
13     int size;
14     DynamicArray(int size) {
15         this->size = size;
16         data = new monomial[size];
17     }
18     DynamicArray(int size, int args[][2]) {
19         this->size = size;
20         data = new monomial[size];
21         for (int i = 0; i < size; i++) {
22             data[i].coeff = args[i][0];
23             data[i].power = args[i][1];

```

C++

```

24     }
25 }
26 DynamicArray(const DynamicArray &other) {
27     size = other.size;
28     data = new monomial[size];
29     for (int i = 0; i < size; i++) {
30         data[i] = other.data[i];
31     }
32 }
33
34 ~DynamicArray() { delete[] data; }
35 };
36
37 class Polynomial {
38     int len = 0;
39     DynamicArray expr{len};
40
41 public:
42     Polynomial(int l) : len(l), expr(l) {}
43     Polynomial(const DynamicArray &other) : len(other.size), expr(other)
44     {}
45
46     void set_product(const Polynomial &a, const Polynomial &b) {
47         if (a.len * b.len == this->len) {
48             for (int i = 0; i < a.len; i++) {
49                 for (int j = 0; j < b.len; j++) {
50                     this->expr.data[i * b.len + j].coeff =
51                         a.expr.data[i].coeff * b.expr.data[j].coeff;
52                     this->expr.data[i * b.len + j].power =
53                         a.expr.data[i].power + b.expr.data[j].power;
54                 }
55             }
56         }
57
58     string format() {
59         string ans;
60         for (int i = 0; i < len; i++) {
61             if (expr.data[i].coeff != 0) {
62                 if (expr.data[i].coeff < 0) {
63                     ans += " - ";
64                 } else if (i > 0) {

```



```

65         ans += " + ";
66     }
67     if (abs(expr.data[i].coeff) != 1 || expr.data[i].power == 0) {
68         ans += to_string(abs(expr.data[i].coeff));
69     }
70     if (expr.data[i].power != 0) {
71         ans += 'x';
72         if (expr.data[i].power != 1) {
73             ans += '^';
74             ans += to_string(expr.data[i].power);
75         }
76     }
77 }
78 }
79 if (ans.size() == 0) {
80     return "0";
81 }
82 return ans;
83 }
84 };
85
86 int main() {
87     int x = 3;
88     int a1[][2] = {{3, 2}, {2, 1}, {1, 0}};
89     DynamicArray d1(x, a1);
90     Polynomial a(d1);
91
92     int y = 2;
93     int a2[][2] = {{3, 2}, {2, 1}};
94     DynamicArray d2(y, a2);
95     Polynomial b(d2);
96
97     int z = x * y;
98     Polynomial c(z);
99     c.set_product(a, b);
100
101     cout << a.format() << endl;
102     cout << "*" << endl;
103     cout << b.format() << endl;
104     cout << '=' << endl;
105     cout << c.format() << endl;
106     return 0;

```

```
107 }
```

Output:

```
1 3x^2 + 2x + 1
2 *
3 3x^2 + 2x
4 =
5 9x^4 + 6x^3 + 6x^3 + 4x^2 + 3x^2 + 2x
```

6. Write a C++ program to implement a Polynomial Calculator using dynamic memory, constructors, copy constructor, move constructor.**Code:**

```
1  #include <iostream>
2
3  using namespace std;
4
5  struct monomial {
6      int coeff;
7      int power;
8  };
9
10 class DynamicArray {
11 public:
12     monomial *data;
13     int size;
14     DynamicArray(int size) {
15         this->size = size;
16         data = new monomial[size];
17     }
18     DynamicArray(int size, int args[][2]) {
19         this->size = size;
20         data = new monomial[size];
21         for (int i = 0; i < size; i++) {
22             data[i].coeff = args[i][0];
23             data[i].power = args[i][1];
24         }
25     }
26     DynamicArray(const DynamicArray &other) {
27         size = other.size;
```

C++

```

28     data = new monomial[size];
29     for (int i = 0; i < size; i++) {
30         data[i] = other.data[i];
31     }
32 }
33
34 ~DynamicArray() { delete[] data; }
35 };
36
37 class Polynomial {
38     int len = 0;
39     DynamicArray expr{len};
40
41 public:
42     Polynomial(int l) : len(l), expr(l) {}
43     Polynomial(const DynamicArray &other) : len(other.size), expr(other)
44     {}
45
46     void set_sum(const Polynomial &a, const Polynomial &b) {
47         if (a.len + b.len == this->len) {
48             for (int i = 0; i < a.len; i++) {
49                 this->expr.data[i] = a.expr.data[i];
50             }
51             for (int i = 0; i < b.len; i++) {
52                 this->expr.data[a.len + i] = b.expr.data[i];
53             }
54         }
55
56     void set_product(const Polynomial &a, const Polynomial &b) {
57         if (a.len * b.len == this->len) {
58             for (int i = 0; i < a.len; i++) {
59                 for (int j = 0; j < b.len; j++) {
60                     this->expr.data[i * b.len + j].coeff =
61                         a.expr.data[i].coeff * b.expr.data[j].coeff;
62                     this->expr.data[i * b.len + j].power =
63                         a.expr.data[i].power + b.expr.data[j].power;
64                 }
65             }
66         }
67
68     string format() {

```

```

69     string ans;
70     for (int i = 0; i < len; i++) {
71         if (expr.data[i].coeff != 0) {
72             if (expr.data[i].coeff < 0) {
73                 ans += " - ";
74             } else if (i > 0) {
75                 ans += " + ";
76             }
77             if (abs(expr.data[i].coeff) != 1 || expr.data[i].power == 0) {
78                 ans += to_string(abs(expr.data[i].coeff));
79             }
80             if (expr.data[i].power != 0) {
81                 ans += 'x';
82                 if (expr.data[i].power != 1) {
83                     ans += '^';
84                     ans += to_string(expr.data[i].power);
85                 }
86             }
87         }
88     }
89     if (ans.size() == 0) {
90         return "0";
91     }
92     return ans;
93 }
94 };
95
96 int main() {
97     int x = 3;
98     int a1[][2] = {{3, 2}, {2, 1}, {1, 0}};
99     DynamicArray d1(x, a1);
100    Polynomial a(d1);
101
102    int y = 2;
103    int a2[][2] = {{3, 2}, {2, 1}};
104    DynamicArray d2(y, a2);
105    Polynomial b(d2);
106
107    char op;
108    cout << "(+)/(*): ";
109    cin >> op;
110

```

```

111 cout << a.format() << endl;
112 cout << op << endl;
113 cout << b.format() << endl;
114 cout << '=' << endl;
115 switch (op) {
116 case '+': {
117     int z = x + y;
118     Polynomial c(z);
119     c.set_sum(a, b);
120     cout << c.format() << endl;
121     break;
122 }
123 case '*': {
124     int z = x * y;
125     Polynomial c(z);
126     c.set_product(a, b);
127     cout << c.format() << endl;
128     break;
129 }
130 default: {
131     cout << "Invalid input." << endl;
132 }
133 }
134 return 0;
135 }

```

Output:

```

1 (+)/(*) : +
2 3x^2 + 2x + 1
3 +
4 3x^2 + 2x
5 =
6 3x^2 + 2x + 1 + 3x^2 + 2x

```

```

1 (+)/(*) : *
2 3x^2 + 2x + 1
3 *
4 3x^2 + 2x
5 =
6 9x^4 + 6x^3 + 6x^3 + 4x^2 + 3x^2 + 2x

```